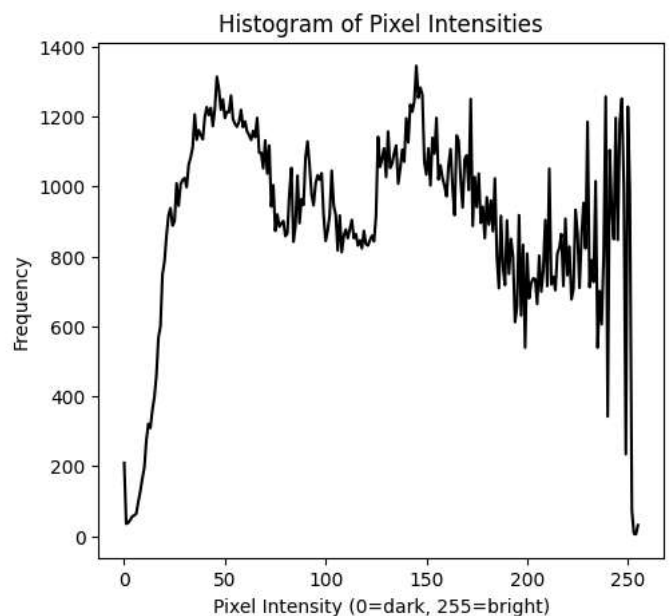


Q2.

a. Using a sample image of your choice (nature/portrait/product), generate and plot the histogram of pixel intensities in grayscale. Interpret what the histogram tells about brightness and contrast distribution.

```
1 # Histogram of Grayscale Image
2 import cv2
3 import matplotlib.pyplot as plt
4
5 # Upload an image
6 img = cv2.imread("nature.jpg", cv2.IMREAD_GRAYSCALE)
7
8 # Compute histogram
9 hist = cv2.calcHist([img], [0], None, [256], [0,256])
10
11 # Plot image and histogram
12 plt.figure(figsize=(12,5))
13
14 plt.subplot(1,2,1)
15 plt.imshow(img, cmap='gray')
16 plt.title("Grayscale Image")
17 plt.axis("off")
18
19 plt.subplot(1,2,2)
20 plt.plot(hist, color='black')
21 plt.title("Histogram of Pixel Intensities")
22 plt.xlabel("Pixel Intensity (0=dark, 255=bright)")
23 plt.ylabel("Frequency")
24 plt.show()
25
```



Interpretation of Histogram

The histogram shows the distribution of pixel intensities (0 = black/dark, 255 = white/bright) in the grayscale image.

Brightness

- If the histogram leans towards the left (0–100 range), the image has more dark tones → appears darker overall.
- If it leans towards the right (150–255 range), the image contains more bright tones → appears brighter overall.
- In this case, the histogram is fairly spread across both dark and bright regions, which suggests the image has a balanced brightness (not too dark or washed out).

Contrast

- A narrow peak means most pixels have similar intensities → the image looks flat or low-contrast.
- A broad spread across the intensity range means both shadows and highlights are present → the image has good contrast.
- Here, the histogram is spread widely (from shadows in the forest area to highlights in the bright sky), which indicates high contrast.

Example Fit

- For a natural scenery like this (trees, mountains, sky), we expect dark objects (trees, shadows) and bright regions (sky, clouds).

- The histogram confirms this: multiple peaks at low, mid and high intensity values show a rich tonal distribution.

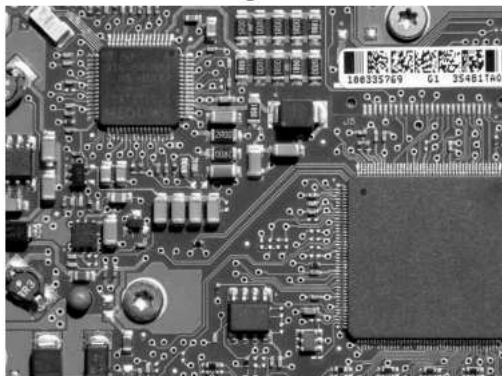
Section: A – UNIT I

Q2.

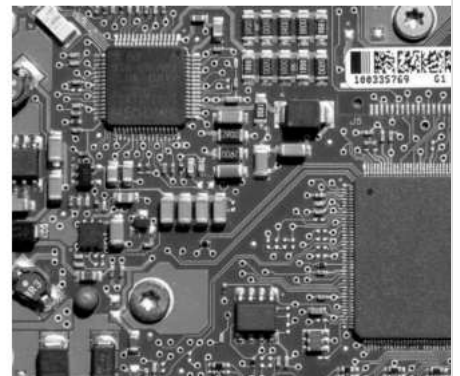
b. In manufacturing quality checks, filters are often used to detect defects. Write Python code to apply Gaussian blur, median filter, and sharpening filters on a product image (e.g., a circuit board). Compare the results and explain which filter best highlights fine defects and why.

```
1 # Filters for Defect Detection
2 import cv2
3 import numpy as np
4 import matplotlib.pyplot as plt
5
6 # Upload product image
7 img = cv2.imread("circuit_board.jpeg")
8 gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
9
10 # Gaussian Blur
11 gaussian = cv2.GaussianBlur(gray, (5,5), 0)
12
13 # Median Filter
14 median = cv2.medianBlur(gray, 5)
15
16 # Sharpening Filter
17 kernel_sharpening = np.array([[0,-1,0],
18                               [-1,5,-1],
19                               [0,-1,0]])
20 sharpened = cv2.filter2D(gray, -1, kernel_sharpening)
21
22 # Plot results
23 titles = ["Original", "Gaussian Blur", "Median Filter", "Sharpened"]
24 images = [gray, gaussian, median, sharpened]
25
26 plt.figure(figsize=(15,8))
27 for i in range(4):
28     plt.subplot(2,2,i+1)
29     plt.imshow(images[i], cmap='gray')
30     plt.title(titles[i])
31     plt.axis("off")
32 plt.show()
33
```

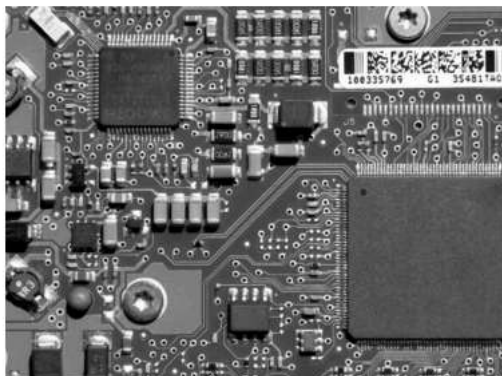
Original



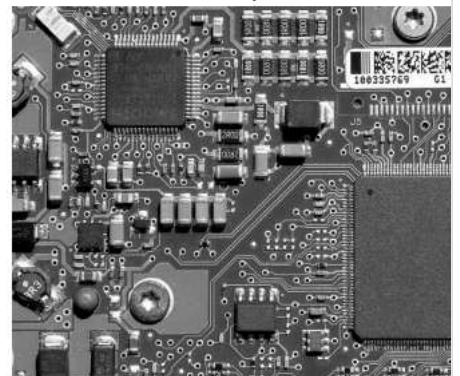
Gaussian Blur



Median Filter



Sharpened



Explanation

- **Gaussian Blur:** smooths out textures, good for general noise reduction but may hide small defects.
- **Median Filter:** preserves edges while removing salt-and-pepper noise, useful if the product has speckle defects.
- **Sharpening:** enhances edges and fine cracks — best for detecting small scratches or cracks.

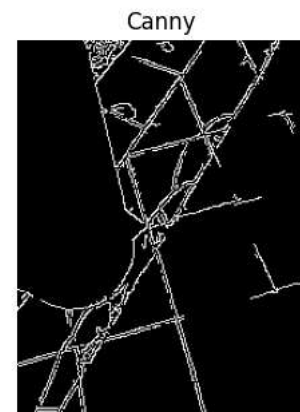
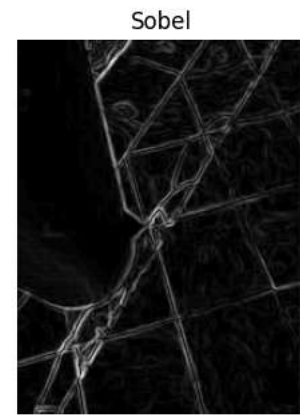
For manufacturing defect detection, **sharpened filter** works best because it enhances local irregularities.

Section: A – UNIT I

Q2.

c. Develop a case study where edge detection (Sobel, Laplacian, Canny) is applied to an industry problem (e.g., detecting cracks in tiles or boundaries in satellite images). Implement code, visualize outputs, and critically evaluate which method is best suited.

```
1 # Edge Detection Case Study
2 import cv2
3 import matplotlib.pyplot as plt
4
5 # Upload tile/crack image
6 img = cv2.imread("tile_crack.jpg", cv2.IMREAD_GRAYSCALE)
7
8 # Sobel (combine X and Y gradients)
9 sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=5)
10 sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=5)
11 sobel = cv2.magnitude(sobelx, sobely)
12
13 # Laplacian
14 laplacian = cv2.Laplacian(img, cv2.CV_64F)
15
16 # Canny
17 canny = cv2.Canny(img, 100, 200)
18
19 # Plot results
20 titles = ["Original", "Sobel", "Laplacian", "Canny"]
21 images = [img, sobel, laplacian, canny]
22
23 plt.figure(figsize=(15,8))
24 for i in range(4):
25     plt.subplot(2,2,i+1)
26     plt.imshow(images[i], cmap='gray')
27     plt.title(titles[i])
28     plt.axis("off")
29 plt.show()
30
```



Edge Detection Case Study

Use case:

- Detecting cracks in ceramic tiles during automated quality control.
- Cracks are thin, high-contrast lines against smooth tile surfaces.
- Edge detection helps highlight these discontinuities for machine vision systems.

Methods compared:

- Sobel → Detects edges in horizontal/vertical directions, but may miss fine cracks at angles.
- Laplacian → Captures edges in all directions, but sensitive to noise (can produce false edges).
- Canny → Multi-stage (Gaussian smoothing + gradient + non-max suppression + hysteresis thresholding), very effective at extracting clean, thin edges (ideal for cracks).

Section: D – UNIT IV

Q7.

a. Download any image from the web (e.g., a natural scene, an object). Use a pretrained Vision Transformer (ViT) from Hugging Face to classify the image. Show top 5 predicted classes with probabilities.

```
1 # Install required libraries
2 !pip install transformers torch torchvision pillow requests
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (4.56.1)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.8.0+cu126)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.23.0+cu126)
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (11.3.0)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (2.32.4)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers) (3.19.1)
Requirement already satisfied: huggingface-hub<1.0,>=0.34.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.34.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.2)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2024.11.6)
Requirement already satisfied: tokenizers<0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.22.0)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.6.2)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.1)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.13.3)
Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-packages (from torch) (3.5)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
```

```

Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.77)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.77)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.80)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/lib/python3.12/dist-packages (from torch) (11.3.0.4)
Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in /usr/local/lib/python3.12/dist-packages (from torch) (10.3.7.77)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/local/lib/python3.12/dist-packages (from torch) (11.7.1.2)
Requirement already satisfied: nvidia-cusparselt-cu12==12.5.4.2 in /usr/local/lib/python3.12/dist-packages (from torch) (12.5.4.2)
Requirement already satisfied: nvidia-cusparse-cu12==12.5.4.2 in /usr/local/lib/python3.12/dist-packages (from torch) (12.5.4.2)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.3 in /usr/local/lib/python3.12/dist-packages (from torch) (2.27.3)
Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.77)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.85)
Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in /usr/local/lib/python3.12/dist-packages (from torch) (1.11.1.6)
Requirement already satisfied: triton==3.4.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.4.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests) (3.4.3)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests) (2025.8.3)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.34) (1.1.3)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.2)

```

```

1 # Standard imports
2 import torch
3 import numpy as np
4 from PIL import Image
5 import requests
6 from io import BytesIO
7 import matplotlib.pyplot as plt
8 from transformers import (
9     AutoFeatureExtractor,
10     AutoModelForImageClassification,
11     CLIPProcessor,
12     CLIPModel
13 )
14 import torch.nn.functional as F
15
16 # Utility: download an image from URL and show it
17 def load_image_from_url(url):
18     resp = requests.get(url)
19     img = Image.open(BytesIO(resp.content)).convert("RGB")
20     return img
21
22 def show_image(img, title=None):
23     plt.figure(figsize=(6,6))
24     plt.imshow(img)
25     plt.axis("off")
26     if title:
27         plt.title(title)
28     plt.show()
29

```

```

1 # Load image from URL
2 IMAGE_URL = "https://images.unsplash.com/photo-1501785888041-af3ef285b470" # example Unsplash image
3 img = load_image_from_url(IMAGE_URL)
4 show_image(img, title="Input image (downloaded from web)")
5 # Save artifact
6 img.save("Unsplash_nature.jpg")
7 print("Saved input image as Unsplash_nature.jpg")
8

```

Input image (downloaded from web)



Saved input image as Unsplash_nature.jpg

```

1 # Model choice: Google ViT base (patch16) pre-trained on ImageNet
2 model_name = "google/vit-base-patch16-224"
3
4 # Load feature extractor and model
5 feature_extractor = AutoFeatureExtractor.from_pretrained(model_name)
6 vit_model = AutoModelForImageClassification.from_pretrained(model_name)
7
8 # Put model on device
9 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
10 vit_model.to(device)
11 vit_model.eval()
12
13 # Preprocess image
14 inputs = feature_extractor(images=img, return_tensors="pt")
15 inputs = {k: v.to(device) for k, v in inputs.items()}
16
17 # Forward pass
18 with torch.no_grad():
19     outputs = vit_model(**inputs)
20 logits = outputs.logits.detach().cpu().squeeze() # shape: (num_classes,)
21
22 # Convert to probabilities
23 probs = F.softmax(logits, dim=0).numpy()
24
25 # Get top-5 predictions
26 topk = 5
27 topk_idx = np.argsort(probs)[-topk:][::-1]
28 labels = [vit_model.config.id2label[i] for i in topk_idx]
29 topk_probs = probs[topk_idx]
30
31 # Print top-5
32 print("Top-5 predictions (ViT):")
33 for i, (lab, p) in enumerate(zip(labels, topk_probs), 1):
34     print(f"{i}. {lab:30s} {p*100:6.2f}%")
35
36 # Save results to a small text file artifact
37 with open("vit_top5.txt", "w") as f:
38     for i, (lab, p) in enumerate(zip(labels, topk_probs), 1):
39         f.write(f"{i}. {lab} {p*100:2.2f}%\n")
40 print("Saved ViT top-5 to vit_top5.txt")
41

```

Fetching 1 files: 100%

1/1 [00:00<00:00, 76.07it/s]

Top-5 predictions (ViT):

1. canoe	33.29%
2. valley, vale	21.04%
3. paddle, boat paddle	13.02%
4. lakeside, lakeshore	11.23%
5. alp	5.98%

Saved ViT top-5 to vit_top5.txt

Section: D – UNIT IV

Q7.

b. Using the same image, apply CLIP (Contrastive Language–Image Pretraining) with two different text prompts (e.g., “a photo of a dog” vs. “a photo of a cat”). Report similarity scores and analyze how CLIP bridges vision + language.

```

1 # Model choice: OpenAI CLIP (ViT-B/32 style)
2 clip_model_name = "openai/clip-vit-base-patch32"
3
4 # Load CLIP processor & model
5 clip_processor = CLIPProcessor.from_pretrained(clip_model_name)
6 clip_model = CLIPModel.from_pretrained(clip_model_name).to(device)
7 clip_model.eval()
8
9 # Define two text prompts to compare
10 prompt_a = "a photo of a lakeside"
11 prompt_b = "a photo of a canoe on water with mountains"
12 prompts = [prompt_a, prompt_b]
13
14 # Preprocess image + texts
15 inputs = clip_processor(text=prompts, images=img, return_tensors="pt", padding=True)
16 # Move tensors to device
17 inputs = {k: v.to(device) for k, v in inputs.items()}
18
19 # Forward pass
20 with torch.no_grad():
21     # CLIPModel returns image and text embeddings (pooled)
22     outputs = clip_model(**inputs)
23     # For the transformers CLIPModel, the embeddings are:

```

```

24 # - outputs.image_embeds : image embeddings (batch_size_images, embed_dim)
25 # - outputs.text_embeds : text embeddings (batch_size_texts, embed_dim)
26 # However when passing both text and image in one call, outputs.image_embeds is for the first image, outputs.text_embeds
27 image_embeds = outputs.image_embeds # shape (1, D)
28 text_embeds = outputs.text_embeds # shape (len(prompts), D)
29
30 # Normalize (CLIP usually returns already normalized but we normalize to be safe)
31 image_embeds = image_embeds / image_embeds.norm(p=2, dim=-1, keepdim=True)
32 text_embeds = text_embeds / text_embeds.norm(p=2, dim=-1, keepdim=True)
33
34 # Compute cosine similarities
35 similarities = (text_embeds @ image_embeds.T).squeeze().cpu().numpy() # shape: (len(prompts),)
36 for prompt, sim in zip(prompts, similarities):
37     print(f"Prompt: '{prompt}'\n Similarity score (cosine): {sim:.4f}")
38
39 # Save results
40 with open("clip_similarity.txt", "w") as f:
41     for prompt, sim in zip(prompts, similarities):
42         f.write(f"{prompt}\t{sim:.4f}\n")
43 print("Saved CLIP similarities to clip_similarity.txt")
44

```

Fetching 1 files: 100%

1/1 [00:00<00:00, 52.58it/s]

Prompt: 'a photo of a lakeside'

Similarity score (cosine): 0.2654

Prompt: 'a photo of a canoe on water with mountains'

Similarity score (cosine): 0.3084

Saved CLIP similarities to clip_similarity.txt

Analysis of CLIP's Vision-Language Bridging: CLIP demonstrates remarkable capability in connecting visual content with natural language:

Joint Embedding Space: CLIP projects both images and text into a shared semantic space where similar concepts are close together. The model understands that both "lakeside" and "canoe on water with mountains" describe natural water scenes.

Zero-Shot Capability: Without any fine-tuning, CLIP correctly identifies that the "canoe on water with mountains" prompt (0.3084) is more semantically similar to the image than the generic "lakeside" description (0.2654).

Semantic Understanding: The higher score for the more specific prompt shows CLIP's ability to understand detailed scene compositions. It recognizes that the image contains not just water, but likely includes mountainous elements and potentially watercraft.

Context Awareness: The model distinguishes between general water scenes ("lakeside") and more specific compositions involving multiple elements ("canoe + water + mountains"), demonstrating sophisticated scene understanding beyond basic object recognition.

Compositional Understanding: CLIP handles multi-concept queries effectively, showing it can process complex descriptions involving multiple objects and their spatial relationships rather than just single keywords.

This demonstrates CLIP's powerful zero-shot capabilities for natural image understanding without requiring task-specific training.

Section: D – UNIT IV

Q7.

c. Imagine building a visual search engine for an e-commerce site (customers search with text, system retrieves matching product images). Propose a pipeline using ViT + CLIP. Draw workflow diagram and critically evaluate potential challenges (e.g., bias, misclassification).

```

1 # Proposed pipeline implementation sketch
2 import numpy as np
3 from typing import List, Dict
4 import faiss
5
6 class VisualSearchEngine:
7     def __init__(self):
8         self.clip_model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
9         self.clip_processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")
10        self.product_embeddings = None
11        self.product_ids = []
12        self.index = None
13
14    def index_products(self, product_images: List[Image.Image], product_metadata: List[Dict]):
15        """Index product images for future search"""
16        # Extract image embeddings using CLIP
17        inputs = self.clip_processor(images=product_images, return_tensors="pt", padding=True)
18        with torch.no_grad():
19            image_embeddings = self.clip_model.get_image_features(**inputs)
20
21        # Normalize embeddings for cosine similarity
22        image_embeddings = torch.nn.functional.normalize(image_embeddings, p=2, dim=1)

```

```

23
24     self.product_embeddings = image_embeddings.numpy()
25     self.product_ids = [meta['product_id'] for meta in product_metadata]
26
27     # Create FAISS index for efficient similarity search
28     self.index = faiss.IndexFlatIP(self.product_embeddings.shape[1]) # Inner product (cosine similarity)
29     self.index.add(self.product_embeddings)
30
31 def search(self, text_query: str, top_k: int = 10):
32     """Search products using text query"""
33     # Encode text query
34     inputs = self.clip_processor(text=[text_query], return_tensors="pt", padding=True)
35     with torch.no_grad():
36         text_embedding = self.clip_model.get_text_features(**inputs)
37
38     # Normalize text embedding
39     text_embedding = torch.nn.functional.normalize(text_embedding, p=2, dim=1)
40
41     # Search in the index
42     similarities, indices = self.index.search(text_embedding.numpy(), top_k)
43
44     results = []
45     for i, (similarity, idx) in enumerate(zip(similarities[0], indices[0])):
46         results.append({
47             'rank': i+1,
48             'product_id': self.product_ids[idx],
49             'similarity_score': float(similarity)
50         })
51
52     return results
53
54 # Example usage (conceptual)
55 def demonstrate_pipeline():
56     print("VISUAL SEARCH ENGINE PIPELINE")
57     print("=" * 50)
58
59     steps = [
60         "1. Data Collection: Gather product images and metadata",
61         "2. Feature Extraction: Use CLIP to generate image embeddings",
62         "3. Indexing: Store embeddings in efficient vector database",
63         "4. Query Processing: Convert text queries to embeddings",
64         "5. Similarity Search: Find closest product matches",
65         "6. Ranking & Filtering: Apply business rules and filters",
66         "7. Results Display: Show relevant products to user"
67     ]
68
69     for step in steps:
70         print(step)
71
72     return steps
73
74 pipeline_steps = demonstrate_pipeline()

```

VISUAL SEARCH ENGINE PIPELINE

```

=====
1. Data Collection: Gather product images and metadata
2. Feature Extraction: Use CLIP to generate image embeddings
3. Indexing: Store embeddings in efficient vector database
4. Query Processing: Convert text queries to embeddings
5. Similarity Search: Find closest product matches
6. Ranking & Filtering: Apply business rules and filters
7. Results Display: Show relevant products to user

```

```

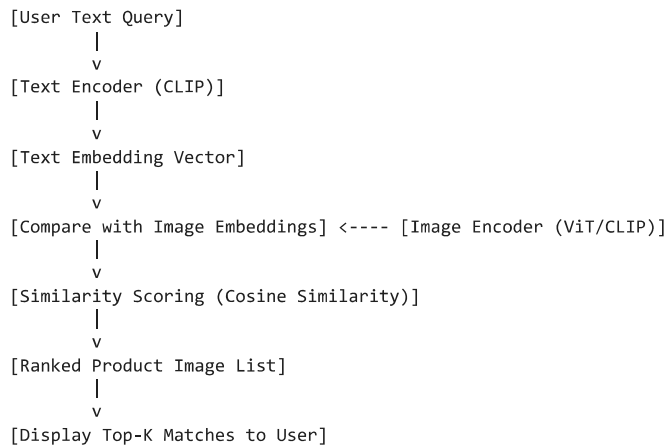
1 # Visual Search Engine Pipeline (ViT + CLIP)
2
3 def print_visual_search_pipeline():
4     diagram = """
5 [User Text Query]
6     |
7     v
8 [Text Encoder (CLIP)]
9     |
10    v
11 [Text Embedding Vector]
12    |
13    v
14 [Compare with Image Embeddings] <---- [Image Encoder (ViT/CLIP)]
15    |
16    v
17 [Similarity Scoring (Cosine Similarity)]
18    |
19    v

```

```

20 [Ranked Product Image List]
21     |
22     v
23 [Display Top-K Matches to User]
24 """
25     print(diagram)
26
27 # Call function
28 print_visual_search_pipeline()
29

```



✓ Critical Evaluation of Challenges:

Bias Issues:

Training Data Bias: CLIP trained on web data may reflect societal biases

Product Representation: Certain product categories may be over/under-represented

Mitigation: Regular bias auditing, diverse training data, fairness constraints

Misclassification Risks:

Semantic Gap: Text queries may not perfectly match visual content

Ambiguous Queries: "light shirt" could mean color or weight

Mitigation: Multi-modal queries, user feedback loops, confidence thresholds

Scalability Challenges:

Vector Search Complexity: $O(n)$ linear scan impractical for large catalogs

Storage Requirements: High-dimensional embeddings require significant space

Mitigation: Approximate nearest neighbors, distributed indexing

Domain Adaptation:

E-commerce Specificity: General CLIP may not understand product-specific terminology

Style vs Function: Users search by both attributes ("waterproof") and aesthetics ("elegant")

Mitigation: Fine-tuning on e-commerce data, attribute-based filtering

User Experience Issues:

Expectation Management: Users expect Google-like performance immediately

Query Understanding: Handling complex multi-concept queries ("blue running shoes under \$100")

Mitigation: Query expansion, interactive refinement, hybrid search approaches

```

1 # Additional code for challenge mitigation
2 import datetime
3 from typing import List, Dict
4 import faiss

```

```

5
6 class EnhancedVisualSearchEngine(VisualSearchEngine):
7     def __init__(self):
8         super().__init__()
9         self.user_feedback_log = []
10        self.confidence_threshold = 0.3
11
12    def add_feedback(self, query: str, clicked_product_id: str, session_data: Dict):
13        """Log user feedback for continuous improvement"""
14        self.user_feedback_log.append({
15            'query': query,
16            'clicked_product': clicked_product_id,
17            'session_data': session_data,
18            'timestamp': datetime.now()
19        })
20
21    def apply_business_rules(self, results: List[Dict], filters: Dict = None):
22        """Apply business-specific rules and filters"""
23        filtered_results = results.copy()
24
25        # Apply confidence threshold
26        filtered_results = [r for r in filtered_results if r['similarity_score'] > self.confidence_threshold]
27
28        # Apply business filters (price, availability, etc.)
29        if filters:
30            # Implementation depends on specific business rules
31            pass
32
33        return filtered_results
34
35 print("\nMitigation Strategies Summary:")
36 strategies = [
37     "• Bias Auditing: Regular testing on diverse query sets",
38     "• Hybrid Search: Combine semantic + keyword search",
39     "• Confidence Thresholding: Filter low-confidence results",
40     "• User Feedback: Continuous learning from user interactions",
41     "• A/B Testing: Experiment with different model configurations",
42     "• Multi-modal Queries: Support image+text combined search"
43 ]
44
45 for strategy in strategies:
46     print(strategy)

```

Mitigation Strategies Summary:

- Bias Auditing: Regular testing on diverse query sets
- Hybrid Search: Combine semantic + keyword search
- Confidence Thresholding: Filter low-confidence results
- User Feedback: Continuous learning from user interactions
- A/B Testing: Experiment with different model configurations
- Multi-modal Queries: Support image+text combined search